3、嵌入式调试方法

嵌入式设备应用开发（嵌入式调试方法）

原创

嵌入式-老费 已于 2024-01-07 21:03:56 修改

阅读量434 收藏

点赞数

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： linux 运维 服务器

基于全志soc的linux嵌入式开发专栏收录该内容

73 篇文章¥49.90¥99.00

已订阅8折续费

[

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益 ›

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

摘要 本文介绍了嵌入式设备的调试方法，包括直接使用gdb、利用gdbserver、基于控制台的调试、日志文件分析、录屏工具辅助以及通过coredump进行调试。调试方法多样，有助于提高开发效率并解决现场问题。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing@163.com】

软件开发不调试，一次性就运行成功，这个概率是很低的。嵌入式开发也是这样的。和一般的x86开发不一样，嵌入式软件开发，还需要把代码编译成指定的cpu指令，转到嵌入式设备上面来执行。这无形之中又增加了调试的难度。当然，如果不涉及到硬件、数据或者具体的协议，直接在x86上面调试还是很方便的。就算涉及到了硬件设备，其实也可以把具体的设备数据转成提前录制好的数据文件，用gdb单步调试也是可以的。

1、直接用gdb调试

pc环境编译和调试，要比交叉编译、调试方便的多。所以，如果不是特别复杂的代码或者片段，我们可以先在pc上面调试，确认没问题之后，再port到嵌入式设备上，这样成功率就会高很多，毕竟可以避免一些低级错误，节省调试时间。比如这样的程序，

```
#include <stdio.h>

int iterate(int data)
{
    if(1 == data)
        return 1;
    else
        return iterate(data-1) + data;
}

int main(int argc, char* argv[])
{
    printf("%d\n", iterate(10));
    return 0;
}
```

下面首先要做的就是把源代码编译成带调试信息的版本，

```
gcc iterate.c -g -o iterate
```

接着就可以用gdb开始调试了，

```
feixiaoxing@feixiaoxing-VirtualBox:~/Desktop/plot$ gdb iterate
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.1) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later
<http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from iterate...
(gdb) b main
Breakpoint 1 at 0x1179: file iterate.c, line 12.
(gdb) c
The program is not being run.
(gdb) r
Starting program: /home/feixiaoxing/Desktop/plot/iterate

Breakpoint 1, main (argc=0, argv=0x7fffffffdee0) at iterate.c:12
12  {
(gdb) quit
```

2、利用gdbserver调试

前面说的调试方法是在pc上面进行调试，其实也是可以利用gdbserver直接在板子上面进行调试的。方法也不复杂，首先是利用交叉编译器生成带调试信息的可执行文件，

```
arm-linux-gnueabi-gcc iterate.c -g -o iterate
```

接着在开发板上面用gdbserver启动程序，

```
gdbserver 192.168.0.99:12345 iterate
```

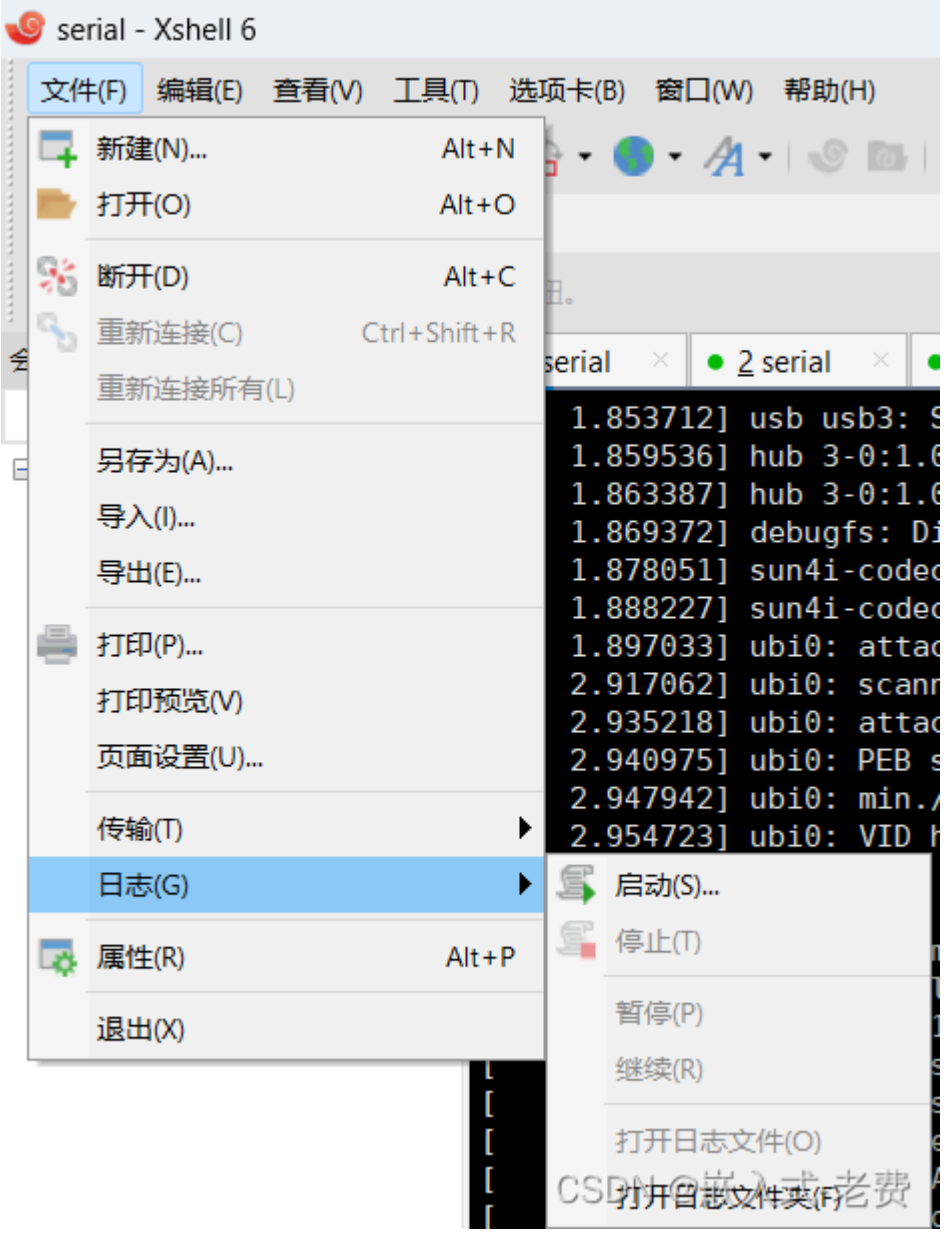
确保gdbserver启动成功之后，就可以用arm-linux-gnueabi-gdb开始调试了，

```
arm-linux-gnueabi-gdb ./iterate
shell> target remote 192.168.0.99:12345
```

这里面有两点需要注意下，第一，gdbserver是运行在开发板上的；第二，arm-linux-gnueabi-gdb是运行在pc上面的。

3、基于控制台的调试

基于控制台的调试也是我们经常遇到的方法。如果是c语言，一般就是printf打印；如果是c++，一般就是std::cout打印，不管是哪一种方式，都需要开发者用控制台启动的方式加载后，我们才能在控制台看到相关的打印。这个时候，我们就可以借助于对应的控制台软件，将打印日志完整地保存下来分析。以xshell为例，



打印之前，首先输入文件-》日志-》启动，这样就可以准备记录日志。等结束之后，再输入文件-》日志-》停止，就可以得到完整的日志打印文件。

4、基于日志文件的调试

很多时候，调试的设备和开发人员是分离开来的。 **但是现场的设备出了问题，就需要技术人员进行分析调试，这个时候就可以通过生成对应的日志文件来分析问题，这样就非常方便和高效。** 常见的参数问题、配置问题、流程问题，都是可以通过日志文件来进行分析处理的。

5、录屏工具的参与

和日志文件一起配合的还是录屏工具。日志文件一般比较抽象一点，而录屏可以让我们第一时间知道现场发生了什么，视频资料会让我们的调试更加地方便、更加地有效率。常见的开源录屏工具有ocam，使用也是很方便的。

6、基于coredump的调试

有一种调试是很麻烦的，那就是程序的闪退。这个时候，上面提到的几种方法就都不好使了，目前剩下的手段就是生成coredump文件，然后借助于gdb、执行文件、coredump来分析程序崩溃的原因，这样才能事半功倍。当然，如果需要生成coredump，一般还需要先输入下面这个命令，

```
ulimit -c unlimited
```

注：

重定向调试也是很好的一种方法，比如./run_app >> log.txt &，这样run_app中的所有日志信息都会转到log.txt当中去了。